Centro Universitário de Excelência
Sistemas de Informação

MecâniQA — Web API Fundacional

Persistência de Dados com MySQL e JDBC — OAT 3

Equipe: São Luiz

Autor: Hellen Verena da Conceição Magalhães
Italo Yan Mendes da Silva

Feira de Santana, 2026

Agenda

O objetivo dessa apresentação é mostrar como a API passou a salvar os dados no MySQL com JDBC puro.

1. Da memória ao MySQL:
O que mudou desde a OAT 1

2. Camadas da API:
O caminho da requisição

3. Repository e JDBC:
SQL escrito à mão

4. Mappers e exceções:
Dados e status HTTP

5. Banco e testes:
MySQL e validação

Da Memória ao MySQL

Duas entregas em memória, uma no banco

OAT 1

Catálogo de peças e serviços

CRUD de /api/pecas e /api/servicos
Repositórios Singleton com ArrayList
Controller chama o Repository direto

✕ **Reiniciou → dados perdidos**

→

OAT 2

Ordens de serviço e pedidos

Status, Builder e itens do pedido
Ainda em ArrayList na memória
Sem Service e sem exceções próprias

✕ **Reiniciou → dados perdidos**

→

OAT 3

Persistência no MySQL

JDBC puro com SQL escrito à mão
Nova camada de Service
Mappers e exceções customizadas

✓ **Dados persistem no banco**

PecaRepository na OAT 1 e 2

```
private final List<Peca> pecas = new ArrayList<>();
```

→

PecaRepository na OAT 3

```
String sql = "SELECT * FROM pecas WHERE codigo = ?";
```

O Caminho da Requisição

Cada camada tem uma única responsabilidade



Requisição enviada pelo Postman

```
GET /api/pecas/1
```

Peça inexistente? → 404 Not Found

Resposta devolvida pelo Controller

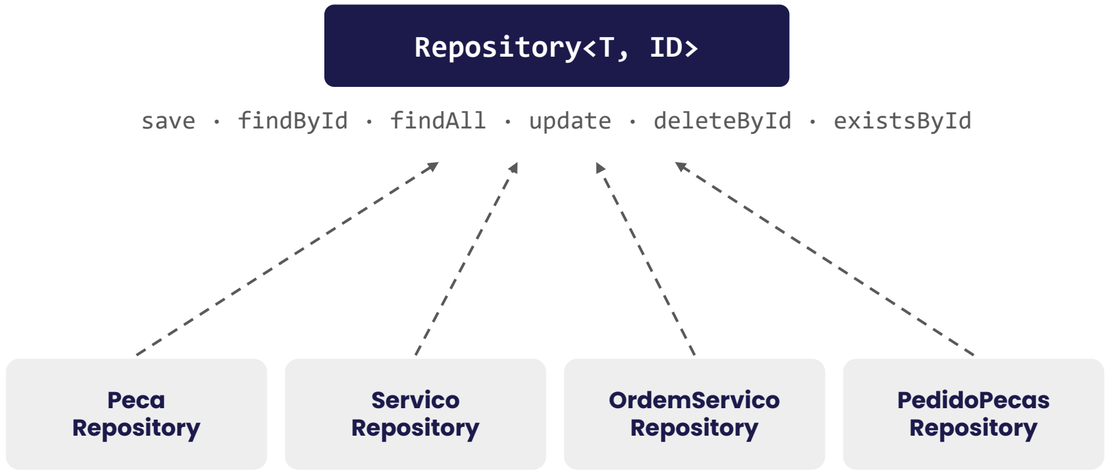
```
200 OK
{
  "codigo": 1,
  "nome": "Filtro de óleo",
  "precoVenda": 39.9,
  "categoria": "MOTOR",
  ...
}
```

Repository e JDBC

Um contrato para todos os repositórios, SQL escrito à mão

1. Abrir conexão `ConexaoMySql.abrir()` → **2. Preparar SQL** `prepareStatement(sql)` → **3. Parâmetros** `setLong(1, codigo)` → **4. Executar** `executeQuery()` → **5. Ler linhas** `resultado.next()` → **6. Converter** `Mapper.toModel(rs)`

Repository<T, ID>: o mesmo CRUD nos 4 repositórios



PecaRepository.findById

```
String sql = "SELECT * FROM pecas WHERE codigo = ?";
PreparedStatement statement = conexao.prepareStatement(sql);
statement.setLong(1, codigo);
ResultSet resultado = statement.executeQuery();
```

1. **`? + setLong()`** — Parâmetros seguros: evita SQL Injection
2. **`try-with-resources`** — Conexão fechada automaticamente

Cliente · Controller · Service · **Repository** · MySQL

Mappers e Exceções

Da tabela ao objeto Java, do erro ao status HTTP

ResultSet Mapper: linha do banco → objeto Java

codigo	nome	preco_venda	categoria
1	Filtro de óleo	39.90	MOTOR

`PecaResultSetMapper.toModel(rs)`

```
Peca { codigo = 1, nome = "Filtro de óleo",
       precoVenda = 39.9, categoria = MOTOR }
```

Um Mapper por entidade · texto → enum · TIMESTAMP → LocalDateTime

Exceções próprias tratadas com try/catch no Controller

`DadosInvalidosException`	**400 Bad Request**
`RecursoNaoEncontradoException`	**404 Not Found**
`Sem exceção`	**200 · 201 · 204**

Sem @ExceptionHandler, como exige o enunciado

Exemplo: GET /api/pecas/99 (peça que não existe)

Repository
findById devolve Optional.empty()

→

Service
lança RecursoNaoEncontradoException

→

Controller
captura no catch

→

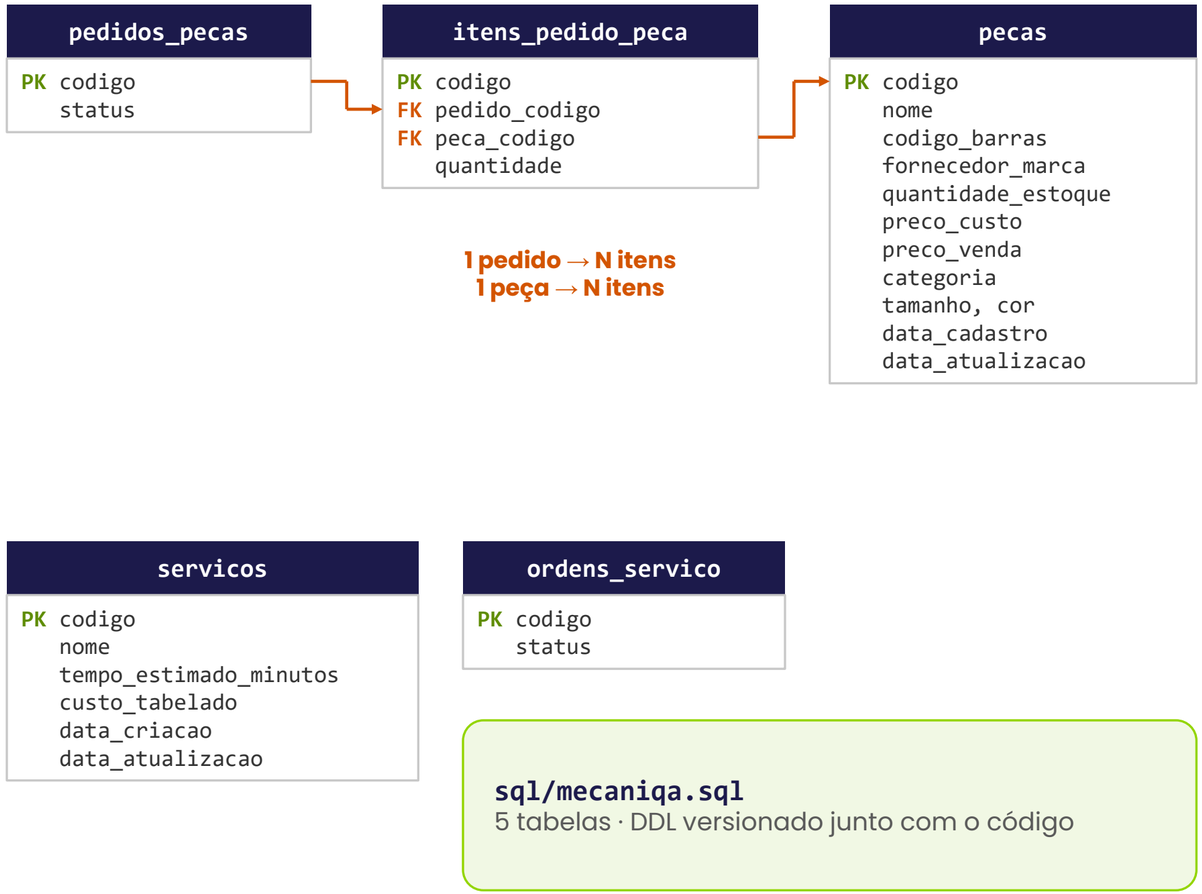
Cliente
recebe 404 Not Found

Cliente | **Controller** | **Service** | **Repository** | MySQL

MySQL e Script SQL

5 tabelas, versionadas junto com o código

- Script em sql/mecaniqa.sql
- Itens ligam pedidos e peças (FK)
- Credenciais em variáveis de ambiente



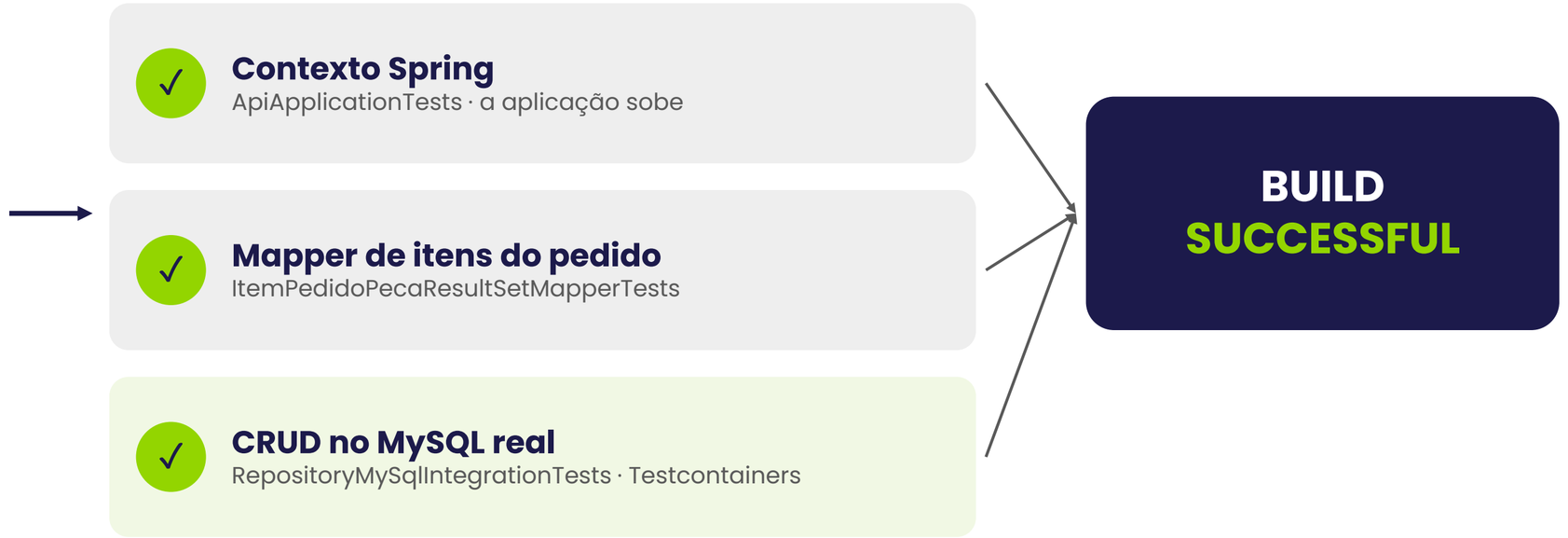
Cliente | Controller | Service | Repository | **MySQL**

Testes e Resultados

Os testes automatizados validam a persistência

Um único comando

```
.\gradlew.bat test
```



O teste de integração cria um MySQL temporário com Docker e executa o CRUD completo dos repositórios. Sem Docker, ele é ignorado e os demais rodam normalmente.

Considerações finais

A jornada completa: cada camada entregou sua parte



Testes automatizados
Mapper de itens do pedido e CRUD completo dos repositórios com Testcontainers, usando um MySQL real.

Aprendizado
Entender na prática o que um ORM faz “por baixo dos panos”: conexão, SQL, conversão dos dados e tratamento de erros.

Referências

- MECÂNIQA. OAT 3 – Persistindo os meus Dados. Programa de Trainee 2026.2.
- ORACLE. JDBC Basics – The Java Tutorials. docs.oracle.com/javase/tutorial/jdbc
- ORACLE. MySQL 8.4 Reference Manual. dev.mysql.com/doc/refman/8.4/en
- SPRING. Spring Boot Reference Documentation. docs.spring.io/spring-boot
- Repositório do projeto: github.com/hellennverenaa/mecaniQA-api-sao-luiz

Centro Universitário de Excelência
Sistemas de Informação

MecâniQA — Web API Fundacional

Persistência de Dados com MySQL e JDBC — OAT 3

Equipe: São Luiz

Autor: Hellen Verena da Conceição Magalhães
Italo Yan Mendes da Silva

Feira de Santana, 2026